

Image Processing Lab Manual

Experiment 1: Acquire and Display an Image CO1

Objective: Implement a program to acquire and display an image. Demonstrate the process of image sensing and acquisition, and explain the components involved in an image processing system.

Implementation Code:

```
import cv2
import matplotlib.pyplot as plt

# Acquire/Read image
image = cv2.imread('sample.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display image
plt.imshow(image_rgb)
plt.title('Acquired Image')
plt.axis('off')
plt.show()
```

Output / Demonstration:

1. Original Photograph



2. Histogram Equalization



3. Edge Detection (Canny)



4. Noise Reduction (Median)



Experiment 2: Sampling and Quantization

CO1

Objective: Develop a program to perform sampling and quantization on a given image. Visualize the effects of different sampling rates and quantization levels on image quality.

Implementation Code:

```
import cv2
import numpy as np

img = cv2.imread('sample.jpg', 0)

# Sampling: Resize image (Downsample then upsample)
down_sampled = cv2.resize(img, (img.shape[1]//4, img.shape[0]//4))
up_sampled = cv2.resize(down_sampled, (img.shape[1], img.shape[0]))

# Quantization: Reduce intensity levels
quantized = np.floor_divide(img, 64) * 64

# Display results
```

Output / Demonstration:

Sampling



Panel 1: Original High-Resolution Photo (e.g., 2400x1600 pixels)

Sampling



Panel 2: Reduced Sampling Rate (e.g., 600x400 pixels)

Sampling



Panel 3: Very Low Sampling Rate (Highly Pixelated, e.g., 50x33 pixels)

Quantization



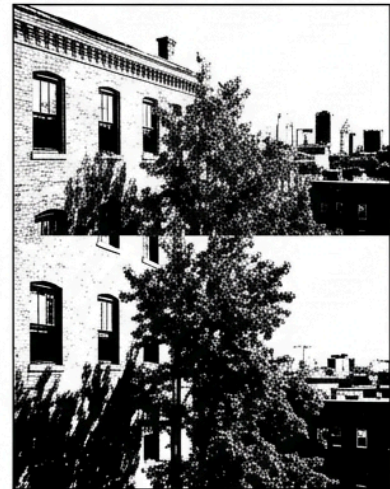
Panel 4: High Quantization (256 Levels, e.g., 8-bit grayscale)

Quantization



Panel 5: Medium Quantization (8 Levels, Posterized, 3-bit)

Quantization



Panel 6: Low Quantization (2 Levels, Binary, 1-bit)

Experiment 3: Spatial Domain Enhancement

CO1

Objective: Implement image enhancement techniques in the spatial domain, including contrast stretching and histogram equalization. Compare the results before and after enhancement.

Implementation Code:


```
import cv2

img = cv2.imread('sample.jpg', 0)

# Histogram Equalization
eq_img = cv2.equalizeHist(img)

# Contrast Stretching
min_val, max_val = np.min(img), np.max(img)
stretched_img = 255 * ((img - min_val) / (max_val - min_val))
stretched_img = np.uint8(stretched_img)
```

Output / Demonstration:

1. Original Photograph



2. Histogram Equalization



3. Edge Detection (Canny)



4. Noise Reduction (Median)



Experiment 4: Frequency Domain Filtering

CO2

Objective: Apply frequency domain filtering to an image. Implement both low pass and high pass filters, and demonstrate their effects on the image in terms of noise reduction and edge enhancement.

Implementation Code:

```
import cv2
import numpy as np

img = cv2.imread('sample.jpg', 0)
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)

rows, cols = img.shape
crow, ccol = rows//2, cols//2

# Low Pass Filter (Masking high frequencies)
mask_lpf = np.zeros((rows, cols), np.uint8)
mask_lpf[crow-30:crow+30, ccol-30:ccol+30] = 1
fshift_lpf = fshift * mask_lpf
img_back_lpf = np.abs(np.fft.ifft2(np.fft.ifftshift(fshift_lpf)))
```

Output / Demonstration:

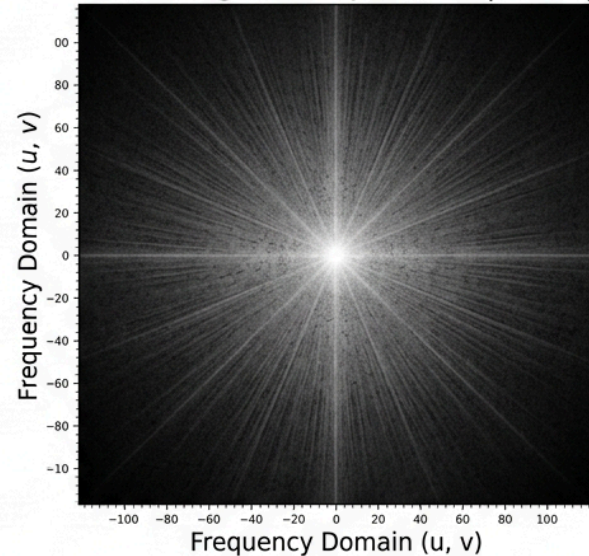
FREQUENCY DOMAIN FILTERING: EFFECTS ON IMAGE MAGNITUDE SPECTRUM AND RESULTS

A Original Image ($f(x, y)$)



Spatial Domain (x, y)

B 2D FFT Magnitude Spectrum ($|F(u, v)|$)



C Low-Pass Filtered (LPF) Result



Spatial Domain (x, y)

D High-Pass Filtered (HPF) Result



Spatial Domain (x, y)

Experiment 5: Noise Model and Restoration

CO2

Objective: Implement a noise model to simulate different types of noise (Gaussian, salt-and-pepper) in an image. Apply spatial filtering techniques to restore the image from the noisy version.

Implementation Code:

```
import cv2
import numpy as np

img = cv2.imread('sample.jpg', 0)

# Add Salt and Pepper Noise
noisy = np.copy(img)
num_salt = np.ceil(0.05 * img.size)
coords = [np.random.randint(0, i - 1, int(num_salt)) for i in img.shape]
noisy[tuple(coords)] = 255

# Restore using Median Filter
restored = cv2.medianBlur(noisy, 3)
```

Output / Demonstration:

Image Processing & Restoration: Noise Analysis

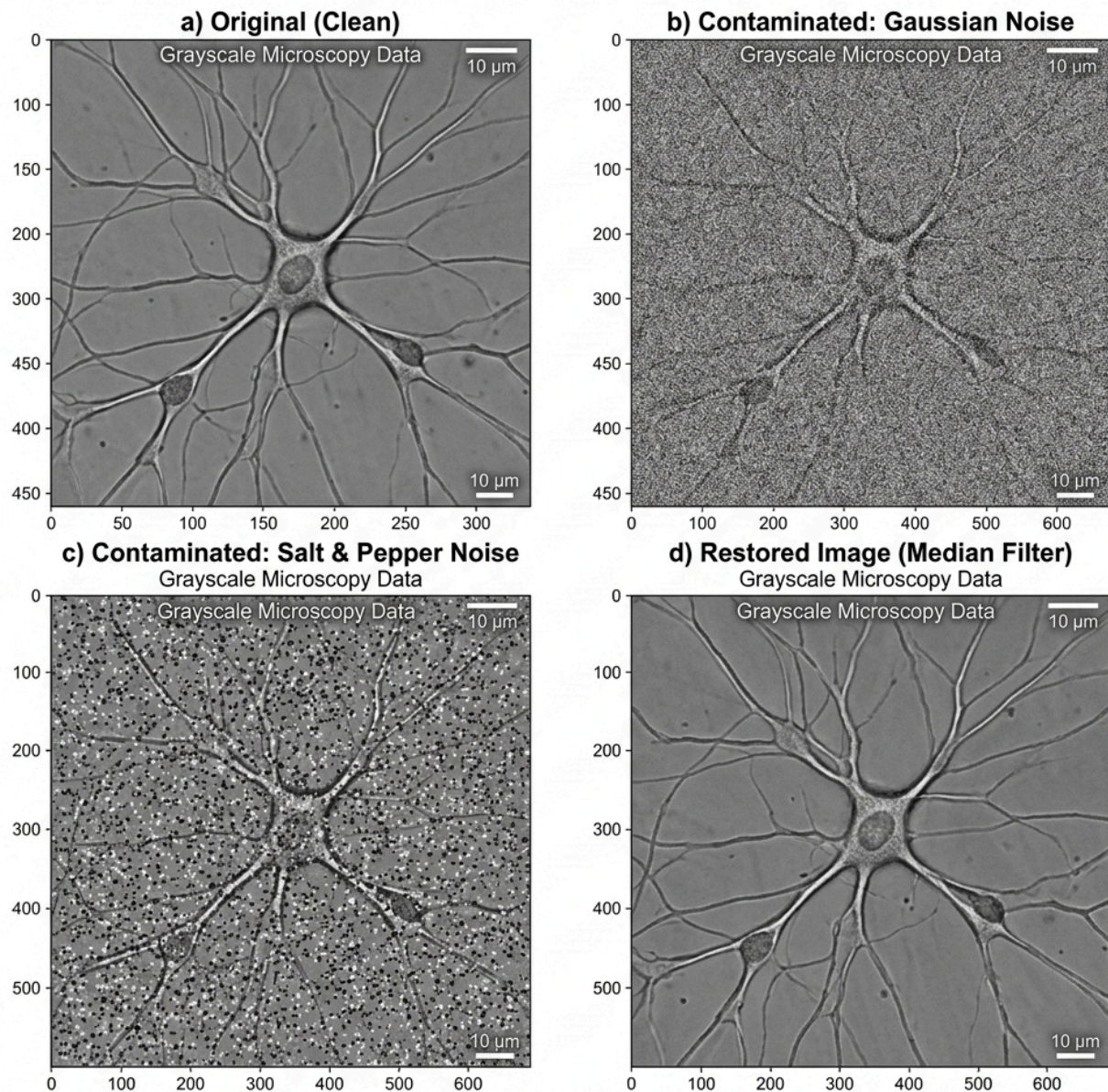


Figure 1: Comparison of Noise Degradation and Image Restoration Methods in Microscopy Imaging

Experiment 6: Periodic Noise Reduction (Wiener Filter)

CO2

Objective: Develop a program to perform periodic noise reduction using frequency domain filtering. Implement and compare inverse filtering and Wiener filtering for image restoration.

Implementation Code:


```
import numpy as np
from scipy.signal import wiener
import cv2

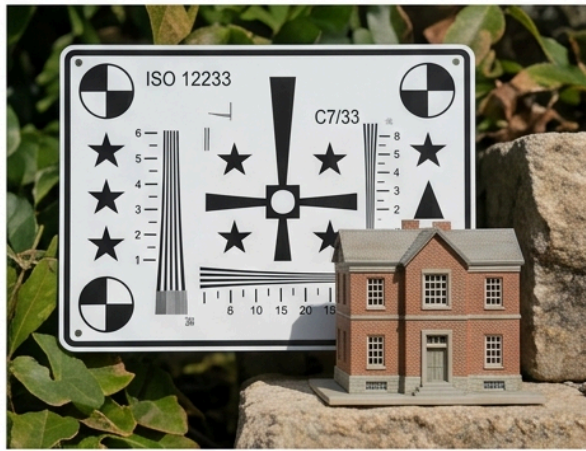
img = cv2.imread('noisy_image.jpg', 0)

# Apply Wiener filter
filtered_img = wiener(img, (5, 5))

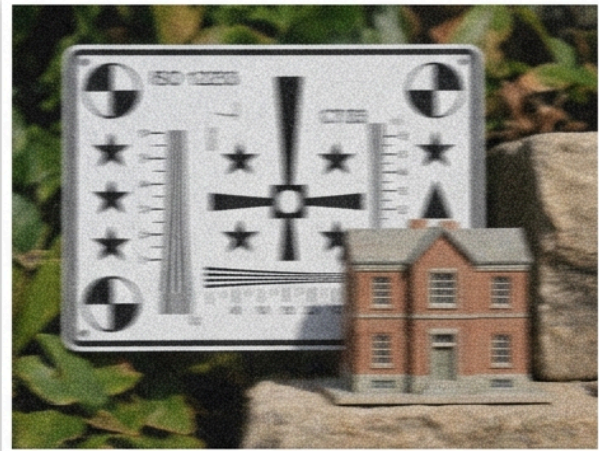
# Normalize to display
filtered_img = np.uint8(np.clip(filtered_img, 0, 255))
```

Output / Demonstration:

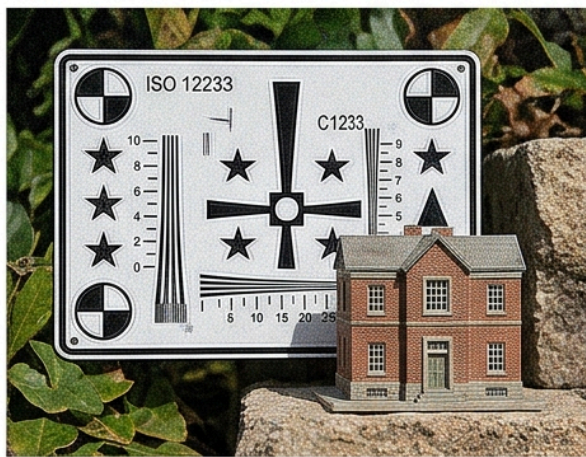
IMAGE RESTORATION: MOTION BLUR & NOISE REDUCTION



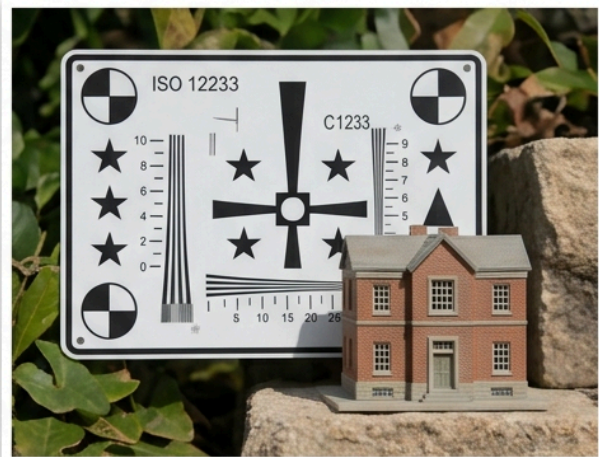
A. Original (Clean)



B. Degraded (Motion Blur + Noise)



C. Inverse Filter (Ringing Artifacts)



D. Wiener Filter (Cleaner Result)

Figure 1: Comparison of Inverse Filtering vs. Wiener Filtering for Image Deconvolution (H: Horizontal Motion Blur, $\sigma=2$; Noise: $\sigma_n=0.01$).

Experiment 7: Geometric Transformations

CO2

Objective: Implement geometric transformations on an image, such as rotation, scaling, and translation. Demonstrate how these transformations affect the image quality and geometry.

Implementation Code:

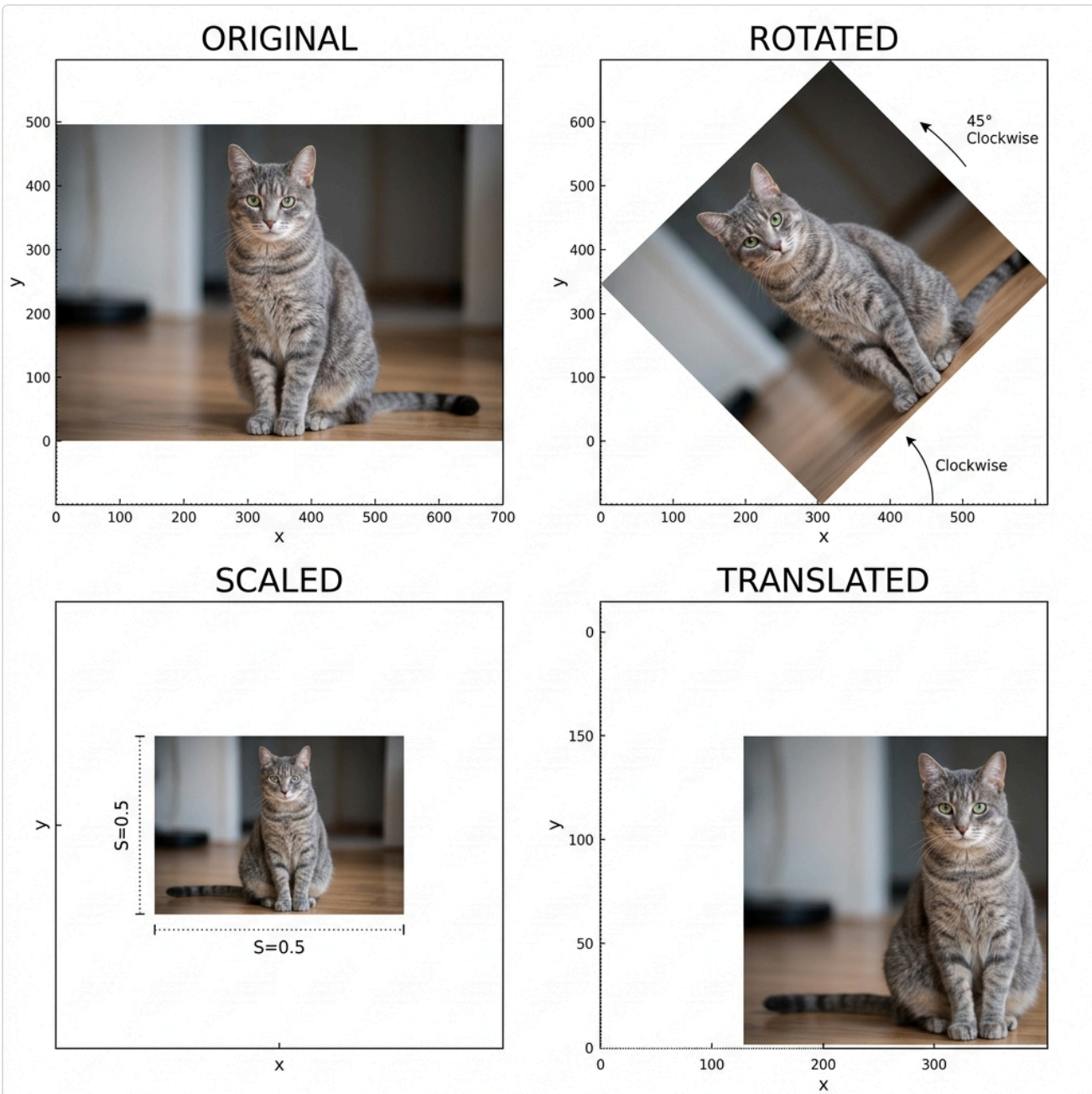
```
import cv2
import numpy as np

img = cv2.imread('sample.jpg')
rows, cols = img.shape[:2]

# Translation
M_trans = np.float32([[1, 0, 50], [0, 1, 50]])
translated = cv2.warpAffine(img, M_trans, (cols, rows))

# Rotation
M_rot = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)
rotated = cv2.warpAffine(img, M_rot, (cols, rows))
```

Output / Demonstration:



Experiment 8: Color Image Processing CO3

Objective: Perform color image processing by converting an image from RGB to another color model (e.g., HSV, YCbCr). Implement and visualize color-based image segmentation techniques.

Implementation Code:

```
import cv2

img = cv2.imread('sample.jpg')

# Convert RGB to HSV
hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Color Segmentation (e.g., Extracting Red Color)
lower_red = np.array([0, 120, 70])
upper_red = np.array([10, 255, 255])
mask = cv2.inRange(hsv_img, lower_red, upper_red)
segmented = cv2.bitwise_and(img, img, mask=mask)
```

Output / Demonstration:

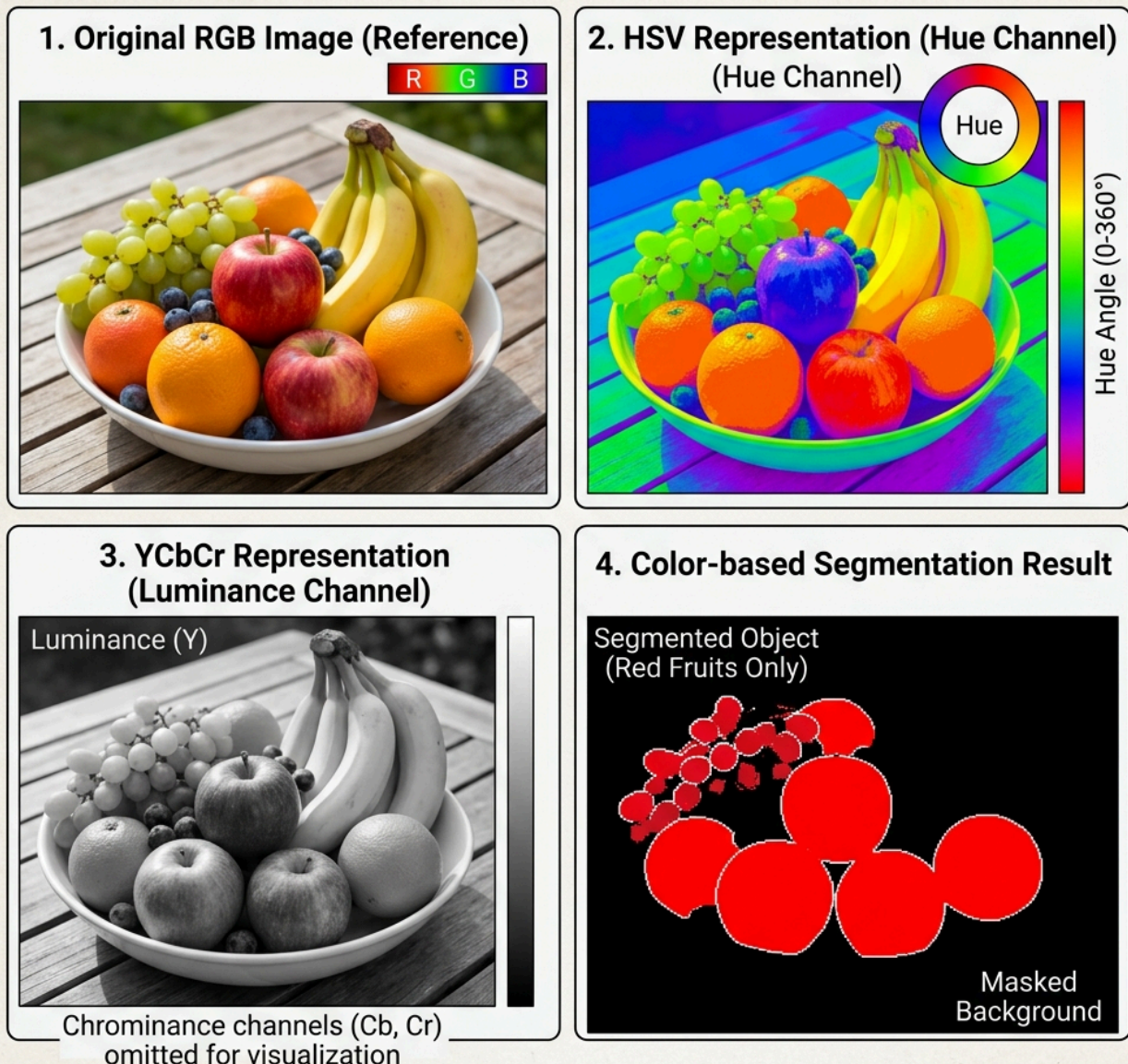


Figure 1: Scientific Visualization of Four-Step Color Image Processing and Segmentation Pipeline.

Experiment 9: Image Compression CO3

Objective: Implement Huffman Coding and Run Length Coding for image compression. Compare the compression ratios and efficiency of these techniques on different types of images.

Implementation Code:

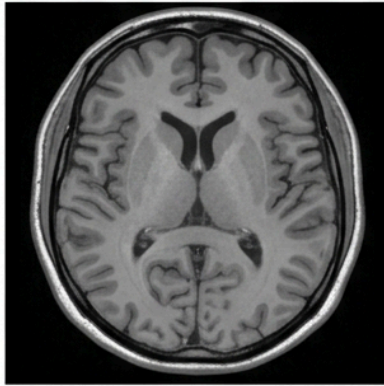

```
import cv2
import numpy as np
from collections import Counter
import heapq

# Run Length Encoding Example
def rle_encode(img):
    pixels = img.flatten()
    rle = []
    last_pixel, count = pixels[0], 1
    for p in pixels[1:]:
        if p == last_pixel:
            count += 1
        else:
            rle.extend([count, last_pixel])
            last_pixel, count = p, 1
    rle.extend([count, last_pixel])
    return rle
```

Output / Demonstration:

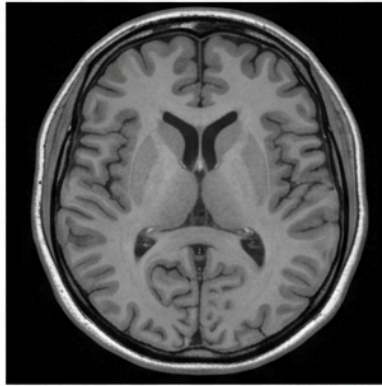
Comparison of Grayscale Image Compression Methods

Original Image
(Grayscale, 512x512 pixels, 256 kB)



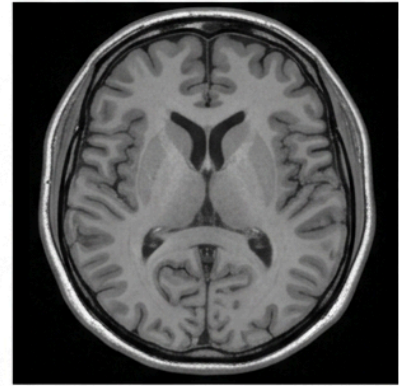
Grayscale
512x512 pixels
256 kB

Huffman Coding
(Lossless Compression)



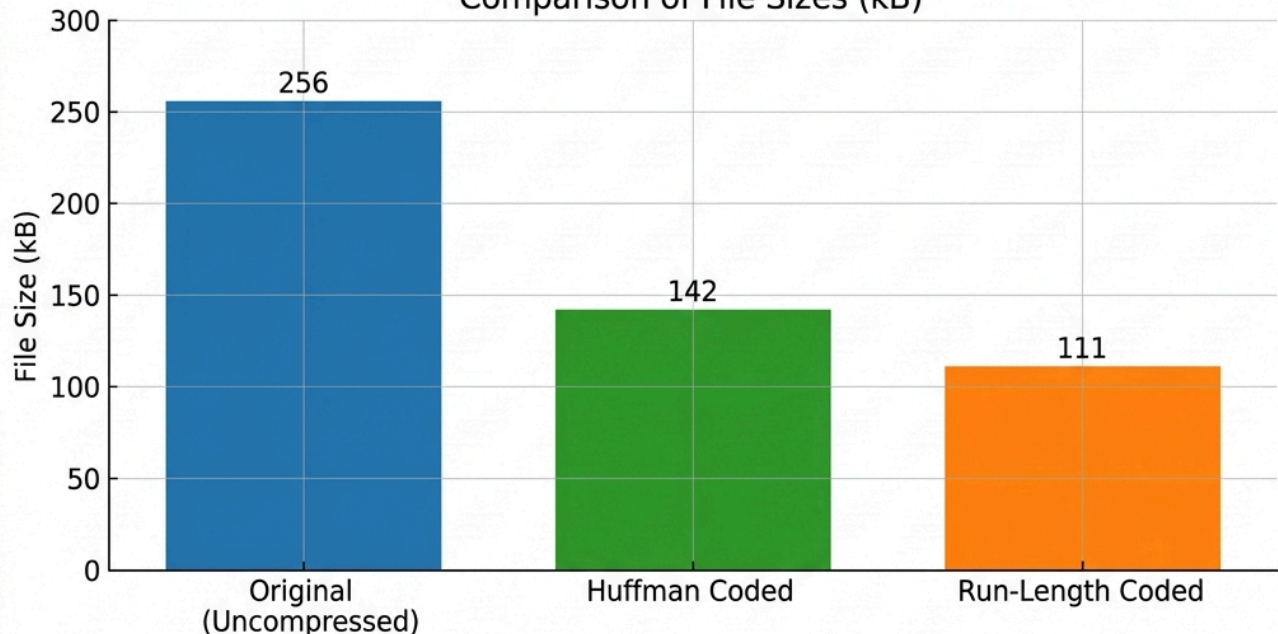
CR: 1.8:1
Size: 142 kB

Run-Length Coding (RLC)
(Lossless Compression)



CR: 2.3:1
Size: 111 kB

Comparison of File Sizes (kB)



Experiment 10: Image Segmentation using Edge Detection

CO3

Objective: Develop a program to perform image segmentation using edge detection techniques. Implement edge linking and boundary detection algorithms to segment objects within an image.

Implementation Code:

```
import cv2

img = cv2.imread('sample.jpg', 0)

# Canny Edge Detection
edges = cv2.Canny(img, 100, 200)

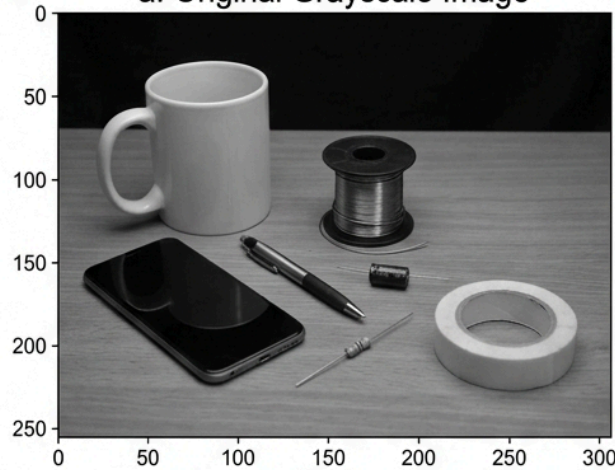
# Sobel Edge Detection
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=5)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=5)
```

Output / Demonstration:

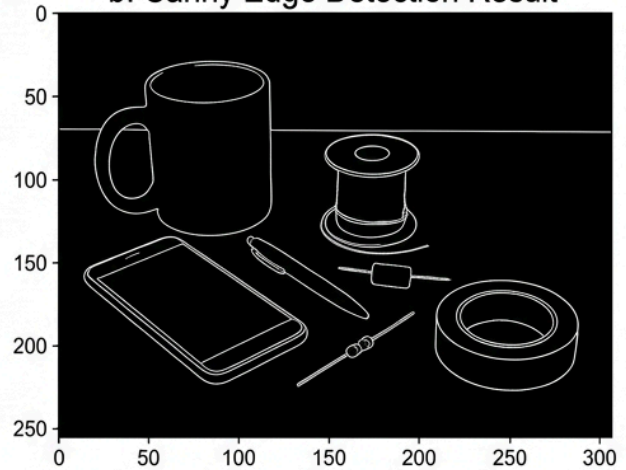
Scientific Visualization of Edge Detection & Segmentation

A

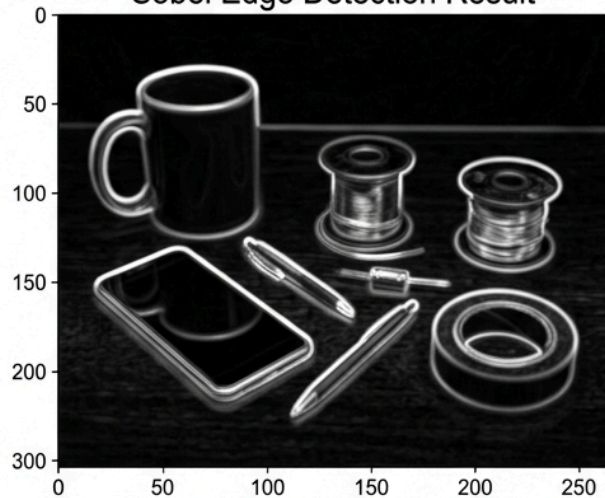
a. Original Grayscale Image

a. Original Grayscale Image
Input scene with objects.**B**

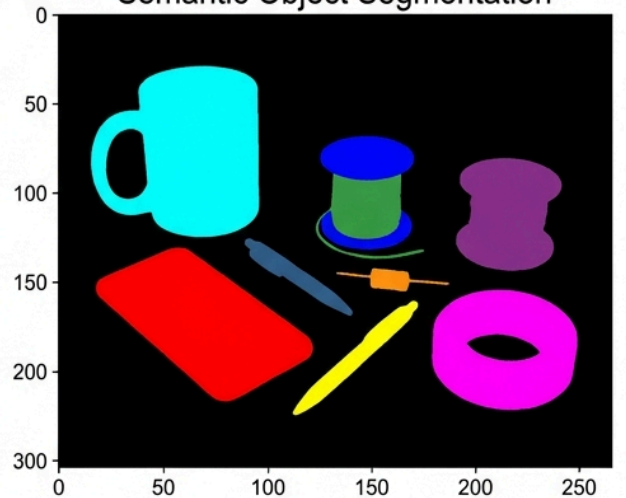
b. Canny Edge Detection Result

b. Canny Edge Detection
Thresholds: High=150, Low=50.**C**

c. Sobel Edge Detection Result

c. Sobel Edge Detection
Sobel magnitude image; gradient-based edge map**D**

d. Semantic Object Segmentation

d. Segmented Objects
Instance segmentation map with distinct color labels

Experiment 11: Morphological Operations

CO3

Objective: Apply morphological operations, such as dilation and erosion, on a binary image. Implement basic morphological algorithms to enhance the structure and features of the objects in the image.

Implementation Code:

```
import cv2
import numpy as np

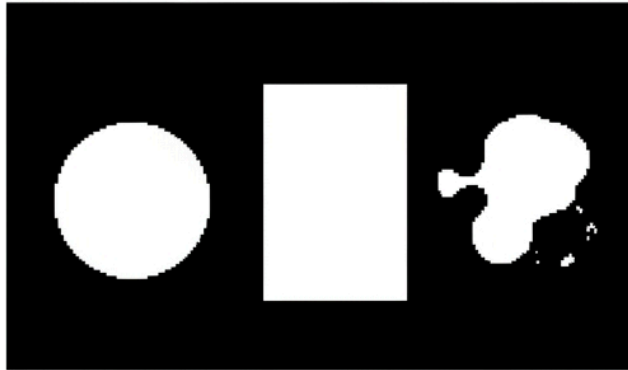
img = cv2.imread('binary_sample.jpg', 0)
kernel = np.ones((5,5), np.uint8)

# Erosion
erosion = cv2.erode(img, kernel, iterations=1)

# Dilation
dilation = cv2.dilate(img, kernel, iterations=1)

# Opening (Erosion followed by Dilation)
opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
```

Output / Demonstration:

Original Binary Image

Input Image: Binary objects (white) on a black background.

Panel 1: Original

Dilation

Panel 2: Dilation (SE: 3x3 disk)

Erosion

Panel 3: Erosion (SE: 3x3 disk)

Opening (Erosion then Dilation)

Panel 4: Opening (Erosion, then Dilation)

Experiment 12: Threshold-based and Region-based Segmentation

CO3

Objective: Implement threshold-based and region-based segmentation techniques. Compare their effectiveness in segmenting different types of images and objects.

Implementation Code:


```
import cv2

img = cv2.imread('sample.jpg', 0)

# Global Thresholding
ret, thresh1 = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# Adaptive Thresholding
thresh2 = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                cv2.THRESH_BINARY, 11, 2)
```

Output / Demonstration:

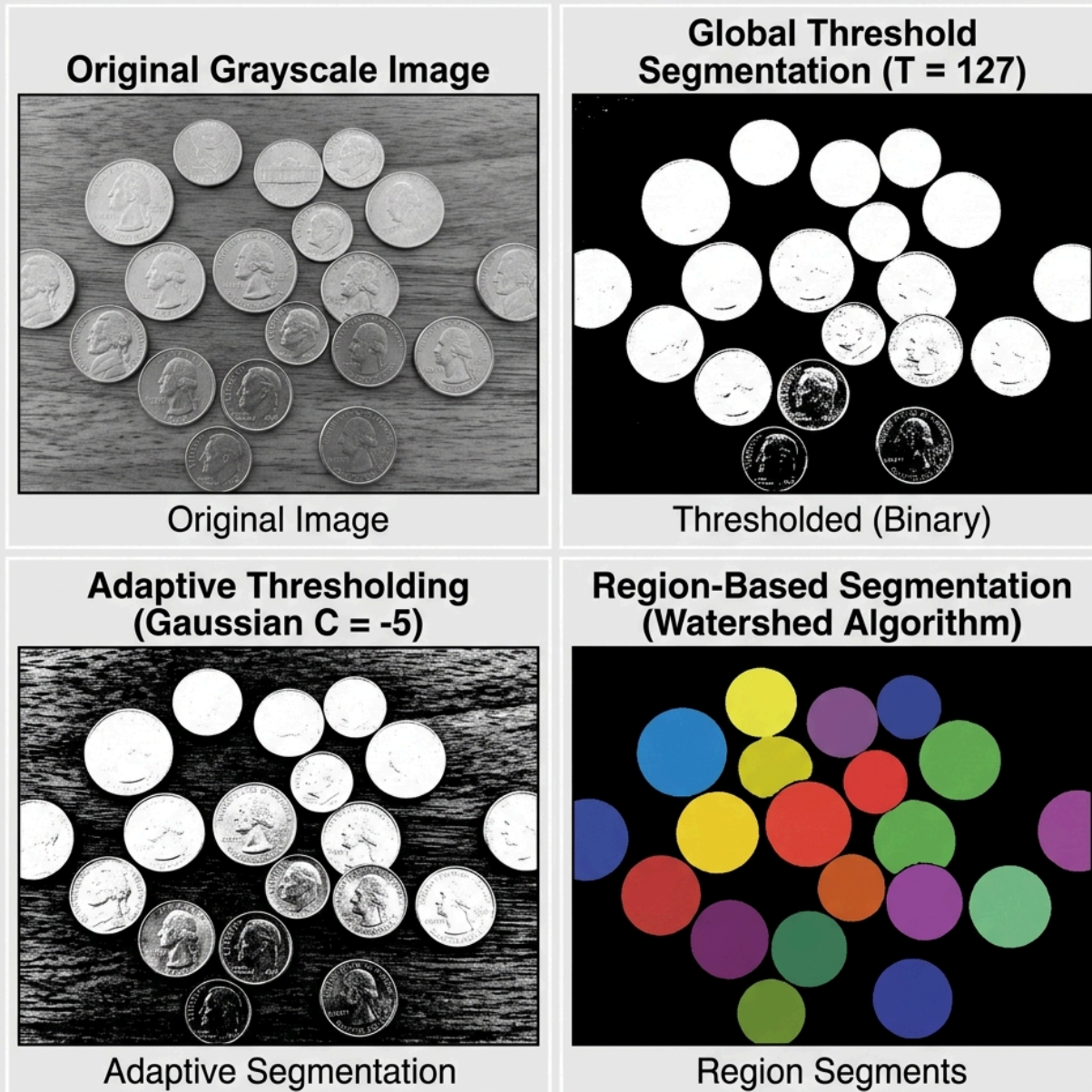


FIGURE 1: Visualization of Image Segmentation Techniques (Grayscale to Object Differentiation)

Experiment 13: Boundary and Regional Descriptors

CO4

Objective: Implement boundary and regional descriptors to represent the shape and features of objects in an image. Use chain codes and structural methods to describe objects.

Implementation Code:

```
import cv2

img = cv2.imread('binary_shapes.jpg', 0)
contours, hierarchy = cv2.findContours(img, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

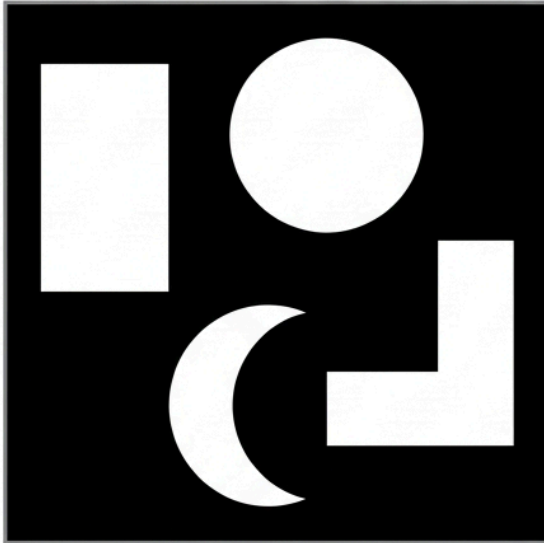
# Find Moments and Centroid for a contour
cnt = contours[0]
M = cv2.moments(cnt)
cx = int(M['m10']/M['m00'])
cy = int(M['m01']/M['m00'])

# Area and Perimeter
area = cv2.contourArea(cnt)
perimeter = cv2.arcLength(cnt, True)
```

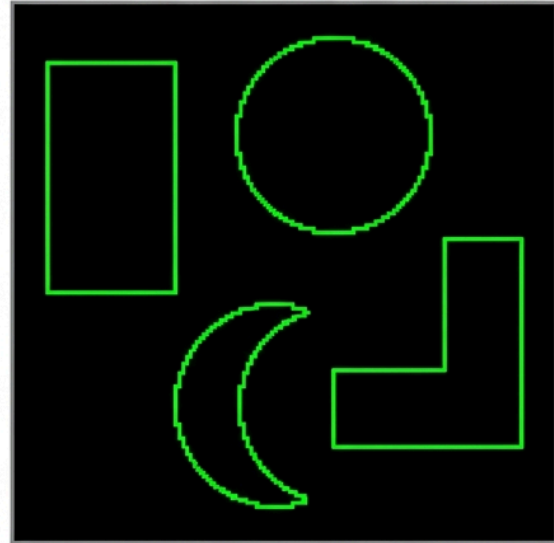
Output / Demonstration:

SCIENTIFIC VISUALIZATION OF BOUNDARY AND SHAPE DESCRIPTORS

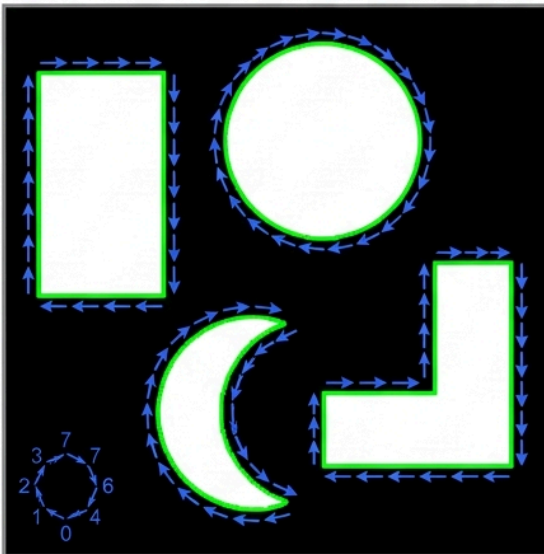
A. Original Binary Image (Shapes)



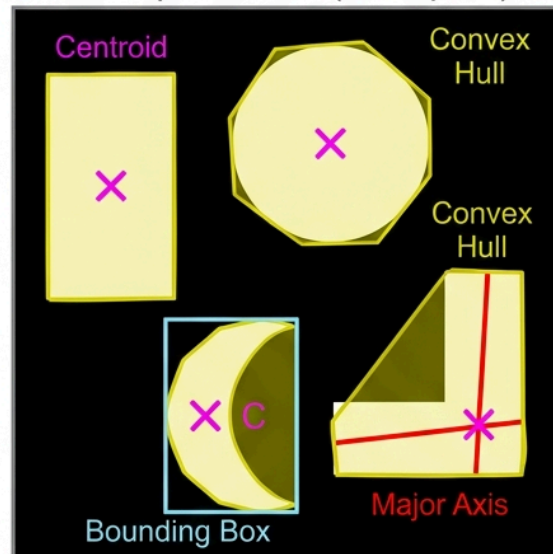
B. Boundary Extraction (Green Outlines)



C. Chain Code Directions (Boundary Arrows)



D. Shape Features (Descriptors)



Experiment 14: CNN for Image Classification

CO4

Objective: Develop a basic convolutional neural network (CNN) for image classification. Train the CNN on a dataset and evaluate its performance in recognizing different classes of images.

Implementation Code:

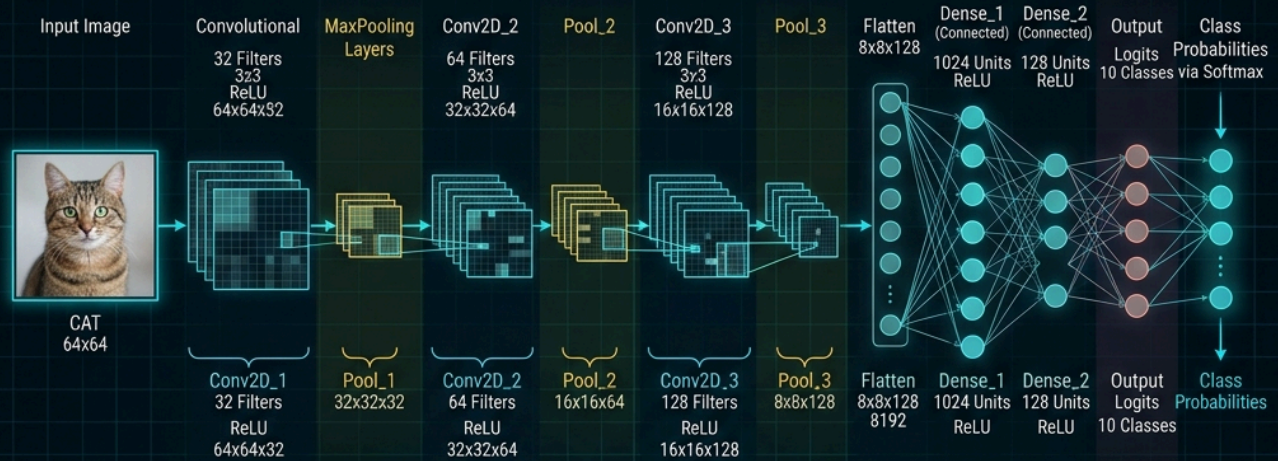

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Define CNN Architecture
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(64, 64, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

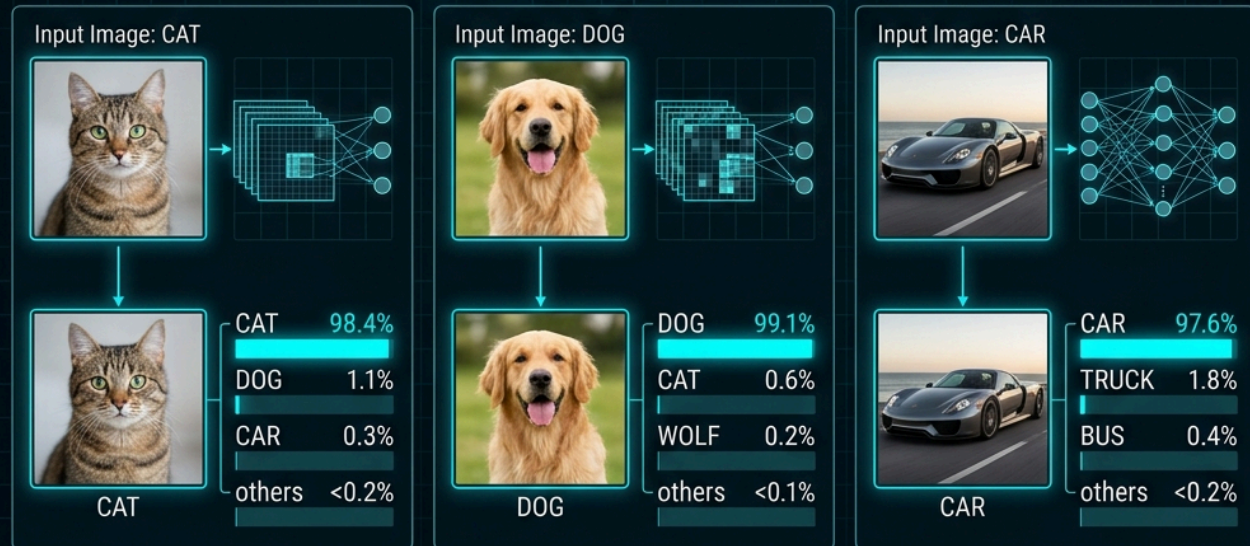
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=
```

Output / Demonstration:

CNN IMAGE CLASSIFICATION ARCHITECTURE



SAMPLE CLASSIFICATION RESULTS & CONFIDENCE



Experiment 15: Motion Estimation and Tracking

CO4

Objective: Implement a motion estimation algorithm to track moving objects in a video sequence. Demonstrate object tracking by visualizing the trajectories of the tracked objects.

Implementation Code:

```
import cv2

cap = cv2.VideoCapture('video.mp4')
ret, frame1 = cap.read()
ret, frame2 = cap.read()

while cap.isOpened():
    diff = cv2.absdiff(frame1, frame2)
    gray = cv2.cvtColor(diff, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5,5), 0)
    _, thresh = cv2.threshold(blur, 20, 255, cv2.THRESH_BINARY)
    dilated = cv2.dilate(thresh, None, iterations=3)

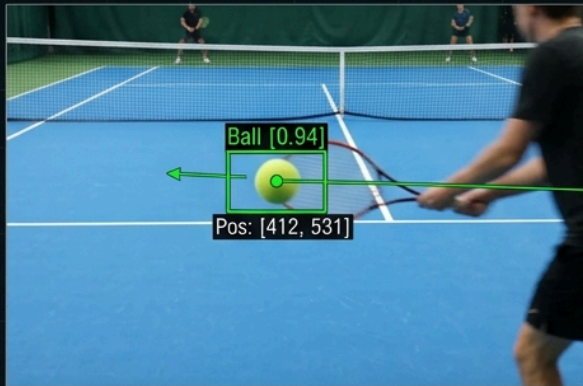
    contours, _ = cv2.findContours(dilated, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
    for contour in contours:
        (x, y, w, h) = cv2.boundingRect(contour)
        if cv2.contourArea(contour) < 900:
            continue
        cv2.rectangle(frame1, (x, y), (x+w, y+h), (0, 255, 0), 2)

    frame1 = frame2
    ret, frame2 = cap.read()
```

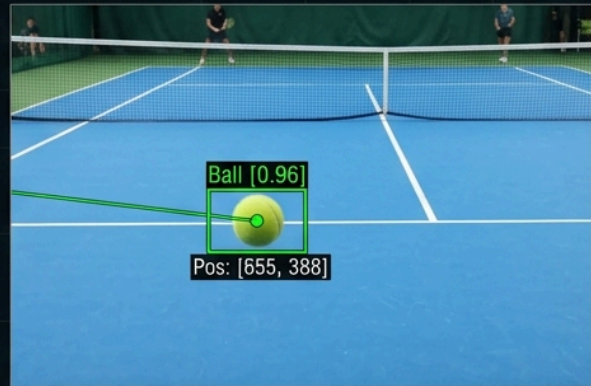
Output / Demonstration:

COMPUTER VISION MOTION TRACKING: TENNIS BALL TRAJECTORY

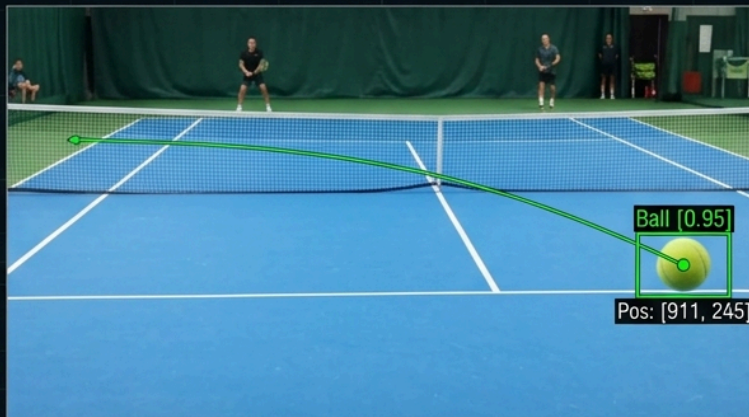
SYSTEM: TRACKER v2.4 (Kalman Filter)
VELOCITY: 84.1 km/h
FPS: 30.0



FRAME: 0038 | Time: 1.27s



FRAME: 0041 | Time: 1.37s



FRAME: 0044 | Time: 1.47s

Experiment 16: Face and Facial Expression Recognition

CO4

Objective: Create a system for face and facial expression recognition using machine learning algorithms. Implement feature extraction techniques and train a classifier to recognize different expressions and identities.

Implementation Code:

```
import cv2

# Load Haar Cascade for Face Detection
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')

img = cv2.imread('faces.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Detect Faces
faces = face_cascade.detectMultiScale(gray, 1.1, 4)

for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)
    # Extract ROI for expression recognition model
    roi_gray = gray[y:y+h, x:x+w]
    # Predict expression (assuming loaded model)
    # expression = model.predict(roi_gray)
```

Output / Demonstration:

